

# Evidence-as-Code Architecture Guide

DevOps.ai

January 2025

## Contents

|                                                                                           |          |
|-------------------------------------------------------------------------------------------|----------|
| <b>1 Evidence-as-Code Architecture Guide</b>                                              | <b>1</b> |
| 1.1 Design Patterns and Implementation Strategies for CI/CD Evidence Automation . . . . . | 1        |
| 1.2 Executive Summary . . . . .                                                           | 1        |
| 1.3 Table of Contents . . . . .                                                           | 2        |
| 1.4 1. Evidence-as-Code Principles . . . . .                                              | 2        |
| 1.5 2. Architecture Patterns . . . . .                                                    | 4        |
| 1.6 3. Cryptographic Signing . . . . .                                                    | 9        |
| 1.7 4. SLSA Provenance . . . . .                                                          | 12       |
| 1.8 5. Retention and Governance . . . . .                                                 | 14       |
| 1.9 6. CI/CD Integration . . . . .                                                        | 16       |
| 1.107. Compliance Framework Mapping . . . . .                                             | 19       |
| 1.118. Implementation Examples . . . . .                                                  | 20       |
| 1.129. Best Practices . . . . .                                                           | 22       |
| 1.1310. Troubleshooting . . . . .                                                         | 22       |

## 1 Evidence-as-Code Architecture Guide

### 1.1 Design Patterns and Implementation Strategies for CI/CD Evidence Automation

Version 1.0 | January 2025 | DevOps.ai

---

### 1.2 Executive Summary

Evidence-as-Code treats compliance evidence as a first-class artifact in the software development lifecycle, automatically generating cryptographically signed, audit-ready evidence bundles from CI/CD pipelines. This approach eliminates manual evidence collection, reduces audit preparation time by 80%, and provides continuous compliance visibility.

This guide provides comprehensive design patterns, implementation strategies, and best practices for building Evidence-as-Code workflows using AIDeci, including crypto-

graphic signing, SLSA provenance, retention policies, and integration with compliance frameworks (SOC 2, ISO 27001, APRA CPS 234, Essential Eight).

**Key Benefits:** - Automate evidence collection from CI/CD pipelines - Generate cryptographically signed evidence bundles - Maintain immutable audit trails with 7-year retention - Reduce audit preparation time from weeks to hours

---

## 1.3 Table of Contents

1. Evidence-as-Code Principles
  2. Architecture Patterns
  3. Cryptographic Signing
  4. SLSA Provenance
  5. Retention and Governance
  6. CI/CD Integration
  7. Compliance Framework Mapping
  8. Implementation Examples
  9. Best Practices
  10. Troubleshooting
- 

## 1.4 1. Evidence-as-Code Principles

### 1.4.1 1.1 Core Principles

**Principle 1: Evidence is Code** - Evidence artifacts are version-controlled alongside application code - Evidence generation is automated in CI/CD pipelines - Evidence schemas are defined as code (JSON Schema, Protobuf)

**Principle 2: Immutability** - Evidence bundles are immutable once signed - Changes require new evidence bundles with provenance links - Audit trails are append-only and tamper-proof

**Principle 3: Cryptographic Trust** - All evidence is cryptographically signed - Signatures are verifiable by auditors - Public keys are published for transparency

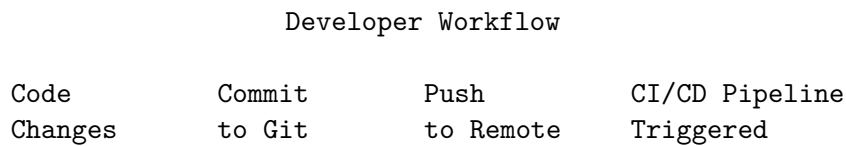
**Principle 4: Continuous Compliance** - Evidence is generated on every build/deployment - Compliance status is visible in real-time - Gaps are detected immediately, not at audit time

**Principle 5: Explainability** - Evidence includes clear rationale for decisions - Audit trails link evidence to requirements - Human-readable reports accompany machine-readable data

### 1.4.2 1.2 Benefits Over Manual Evidence Collection

| Aspect               | Manual Collection       | Evidence-as-Code           |
|----------------------|-------------------------|----------------------------|
| <b>Time</b>          | 10-20 days per audit    | 2-4 hours per audit        |
| <b>Accuracy</b>      | Human error prone       | Automated, consistent      |
| <b>Timeliness</b>    | Collected at audit time | Continuous generation      |
| <b>Completeness</b>  | May miss evidence       | Comprehensive coverage     |
| <b>Verifiability</b> | Unsigned documents      | Cryptographically signed   |
| <b>Retention</b>     | Manual archiving        | Automated 7-year retention |
| <b>Cost</b>          | High labor cost         | Low operational cost       |

### 1.4.3 1.3 Evidence-as-Code Workflow



Build & Test  
(CI/CD Pipeline)

Generate Evidence

- SBOM
- SARIF scans
- Test results
- Build metadata

Push to AlDeci

- Normalize
- Correlate
- Score risk
- Evaluate policy

Generate Evidence  
Bundle

- Sign with RSA
- Add provenance

- Store 7 years

Publish Evidence

- S3 storage
  - Jira attachment
  - Audit dashboard
- 

## 1.5 2. Architecture Patterns

### 1.5.1 2.1 Pattern 1: Pipeline-Embedded Evidence Generation

**Description:** Evidence generation is embedded directly in CI/CD pipeline as a build step.

**Use Case:** Greenfield projects with full control over CI/CD pipelines.

**Architecture:**

```
# .github/workflows/build.yml
name: Build and Generate Evidence

on: [push]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Build application
        run: npm run build

      - name: Generate SBOM
        run: syft . -o cyclonedx-json > sbom.json

      - name: Run SAST
        run: semgrep --sarif > sarif.json

      - name: Run tests
        run: npm test -- --coverage

      - name: Push evidence to AlDeci
        run: |
          curl -X POST https://api.devops.ai/products/aldeci/inputs/sbom \
            -H "Authorization: Bearer ${ secrets.ALDECI_TOKEN }" \
```

```

-F "file=@sbom.json"

curl -X POST https://api.devops.ai/products/aldeci/inputs/sarif \
-H "Authorization: Bearer ${ secrets.ALDECI_TOKEN }}" \
-F "file=@sarif.json"

- name: Generate evidence bundle
  run: |
    curl -X POST https://api.devops.ai/products/aldeci/evidence/generate \
    -H "Authorization: Bearer ${ secrets.ALDECI_TOKEN }}" \
    -H "Content-Type: application/json" \
    -d '{
      "component": "my-app",
      "version": "${ github.sha }",
      "compliance_frameworks": ["soc2", "iso27001"]
    }' \
    -o evidence-bundle.zip

- name: Upload evidence bundle
  uses: actions/upload-artifact@v3
  with:
    name: evidence-bundle
    path: evidence-bundle.zip
    retention-days: 2555 # 7 years

```

**Pros:** - Simple to implement - Evidence generated on every build - No additional infrastructure required

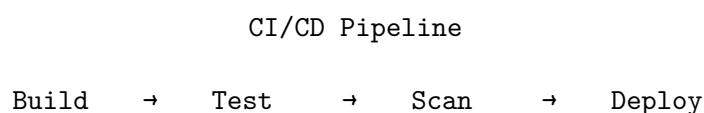
**Cons:** - Couples evidence generation to build pipeline - May slow down build times - Requires pipeline modification for each project

### 1.5.2 2.2 Pattern 2: Sidecar Evidence Collector

**Description:** Evidence collection runs as a sidecar process that monitors CI/CD artifacts.

**Use Case:** Brownfield projects with existing CI/CD pipelines that can't be easily modified.

**Architecture:**



Artifact Storage  
(S3, Artifactory)

Evidence Collector  
(Sidecar Process)  
- Watch for artifacts  
- Push to AlDeci  
- Generate bundles

AlDeci  
- Normalize  
- Correlate  
- Generate evidence

## Implementation:

```
# evidence_collector.py
import boto3
import requests
import time

s3 = boto3.client('s3')
aldecapi_api = "https://api.devops.ai/products/aldecapi"
aldecapi_token = os.getenv("ALDECAPI_TOKEN")

def watch_artifacts():
    """Watch S3 bucket for new artifacts and push to AlDeci"""
    while True:
        # List new artifacts
        response = s3.list_objects_v2(
            Bucket='ci-artifacts',
            Prefix='builds/',
            StartAfter=last_processed_key
        )

        for obj in response.get('Contents', []):
            key = obj['Key']

            # Download artifact
```

```

        artifact = s3.get_object(Bucket='ci-artifacts', Key=key)

        # Determine artifact type
        if key.endswith('.sbom.json'):
            push_sbom(artifact['Body'].read())
        elif key.endswith('.sarif'):
            push_sarif(artifact['Body'].read())
        elif key.endswith('.test-results.json'):
            push_test_results(artifact['Body'].read())

        last_processed_key = key

        time.sleep(60) # Check every minute

def push_sbom(sbom_data):
    """Push SBOM to AlDeci"""
    response = requests.post(
        f"{aldec_api}/inputs/sbom",
        headers={"Authorization": f"Bearer {aldec_token}"},
        files={"file": sbom_data}
    )
    response.raise_for_status()

def push_sarif(sarif_data):
    """Push SARIF to AlDeci"""
    response = requests.post(
        f"{aldec_api}/inputs/sarif",
        headers={"Authorization": f"Bearer {aldec_token}"},
        files={"file": sarif_data}
    )
    response.raise_for_status()

```

**Pros:** - No modification to existing pipelines - Centralized evidence collection - Can collect from multiple CI/CD systems

**Cons:** - Additional infrastructure required - Potential delay in evidence generation - Requires artifact storage integration

### 1.5.3 2.3 Pattern 3: GitOps Evidence Reconciliation

**Description:** Evidence generation triggered by GitOps reconciliation loops (ArgoCD, Flux).

**Use Case:** Kubernetes-native applications using GitOps deployment patterns.

**Architecture:**

Git Repository

```
manifests/  
  deployment.yaml  
  service.yaml  
  evidence-policy.yaml ← Evidence requirements
```

```
ArgoCD / Flux  
- Sync manifests  
- Deploy to K8s  
- Trigger evidence
```

```
Evidence Controller  
(K8s Operator)  
- Watch deployments  
- Generate SBOM  
- Scan images  
- Push to AlDeci
```

```
AlDeci  
- Generate evidence  
- Validate policy  
- Store bundles
```

## Implementation:

```
# evidence-policy.yaml  
apiVersion: evidence.devops.ai/v1  
kind: EvidencePolicy  
metadata:  
  name: my-app-evidence  
  namespace: production  
spec:  
  component: my-app  
  version: v2.1.0  
  complianceFrameworks:  
    - soc2  
    - iso27001  
  evidenceRequirements:
```



```

- type: sbom
  format: cyclonedx
  required: true
- type: sarif
  scanTypes: [SAST, SCA, container]
  required: true
- type: provenance
  format: slsa-v1
  required: true
retentionDays: 2555 # 7 years
signing:
  algorithm: RSA-SHA256
  keyRef: evidence-signing-key

```

**Pros:** - Native Kubernetes integration - Declarative evidence requirements - Automatic evidence on every deployment

**Cons:** - Requires Kubernetes - Additional operator deployment - More complex setup

---

## 1.6 3. Cryptographic Signing

### 1.6.1 3.1 Signing Algorithms

AlDeci supports two signing algorithms:

**RSA-SHA256** (Recommended for most use cases) - Key size: 2048-bit or 4096-bit - Signature size: 256 bytes (2048-bit) or 512 bytes (4096-bit) - Performance: ~1ms per signature - Compatibility: Widely supported, auditor-friendly

**Cosign** (Recommended for container images) - Based on Sigstore project - Keyless signing with OIDC - Transparency log integration - Container-native

### 1.6.2 3.2 Key Management

#### Key Generation:

```

# Generate RSA key pair
openssl genrsa -out evidence-signing-key.pem 4096
openssl rsa -in evidence-signing-key.pem -pubout -out evidence-signing-key.pub

# Store private key securely
aws secretsmanager create-secret \
  --name aldecidevidence-signing-key \
  --secret-string file://evidence-signing-key.pem

# Publish public key for verification
aws s3 cp evidence-signing-key.pub s3://public-keys/aldecidevidence-signing-key.pub --acl publ

```

## Key Rotation:

```
# Generate new key pair
openssl genrsa -out evidence-signing-key-2025.pem 4096
openssl rsa -in evidence-signing-key-2025.pem -pubout -out evidence-signing-key-2025.pub

# Update AlDeci configuration
curl -X PUT https://api.devops.ai/products/aldecide/config/signing \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "algorithm": "RSA-SHA256",
    "key_id": "2025-01",
    "key_path": "/app/keys/evidence-signing-key-2025.pem",
    "public_key_url": "https://public-keys.s3.amazonaws.com/aldecide/evidence-signing-key-2025.p
  }'

# Old key remains valid for verification
# New evidence signed with new key
# Rotation complete
```

**Key Storage Best Practices:** - Store private keys in AWS Secrets Manager, Azure Key Vault, or HashiCorp Vault - Never commit private keys to Git - Use IAM roles for key access (no hardcoded credentials) - Enable key rotation every 12 months - Maintain old keys for verification of historical evidence

### 1.6.3 3.3 Signature Verification

#### Manual Verification:

```
# Download evidence bundle
wget https://evidence.devops.ai/bundles/evidence-20250126-abc123.zip

# Extract bundle
unzip evidence-20250126-abc123.zip

# Download public key
wget https://public-keys.s3.amazonaws.com/aldecide/evidence-signing-key.pub

# Verify signature
openssl dgst -sha256 -verify evidence-signing-key.pub \
  -signature signatures/manifest.sig \
  manifest.yaml

# Output: Verified OK
```

#### Automated Verification:

```
# verify_evidence.py
import hashlib
```

```

from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.backends import default_backend

def verify_evidence_bundle(bundle_path, public_key_url):
    """Verify evidence bundle signature"""

    # Extract bundle
    with zipfile.ZipFile(bundle_path, 'r') as zip_ref:
        zip_ref.extractall('evidence')

    # Read manifest
    with open('evidence/manifest.yaml', 'rb') as f:
        manifest_data = f.read()

    # Read signature
    with open('evidence/signatures/manifest.sig', 'rb') as f:
        signature = f.read()

    # Download public key
    response = requests.get(public_key_url)
    public_key = serialization.load_pem_public_key(
        response.content,
        backend=default_backend()
    )

    # Verify signature
    try:
        public_key.verify(
            signature,
            manifest_data,
            padding.PKCS1v15(),
            hashes.SHA256()
        )
        return True
    except Exception as e:
        print(f"Verification failed: {e}")
        return False

# Usage
if verify_evidence_bundle('evidence-20250126-abc123.zip', 'https://public-keys.s3.amazonaws.com'):
    print("Evidence bundle verified successfully")
else:
    print("Evidence bundle verification failed")

```

## 1.7 4. SLSA Provenance

### 1.7.1 4.1 SLSA Overview

SLSA (Supply-chain Levels for Software Artifacts) is a framework for ensuring software supply chain integrity. AlDeci generates SLSA v1 provenance attestations for all evidence bundles.

**SLSA Levels:** - **SLSA 1:** Provenance exists (build metadata documented) - **SLSA 2:** Provenance is signed (tamper-proof) - **SLSA 3:** Provenance is non-forgable (isolated build environment) - **SLSA 4:** Provenance is auditable (two-party review)

AlDeci provides **SLSA Level 2** provenance by default (signed provenance).

### 1.7.2 4.2 Provenance Attestation Structure

```
{
  "_type": "https://in-toto.io/Statement/v0.1",
  "subject": [
    {
      "name": "evidence-20250126-abc123.zip",
      "digest": {
        "sha256": "a1b2c3d4e5f6..."
      }
    }
  ],
  "predicateType": "https://slsa.dev/provenance/v1",
  "predicate": {
    "buildDefinition": {
      "buildType": "https://devops.ai/aldeci/evidence-generation/v1",
      "externalParameters": {
        "component": "my-app",
        "version": "v2.1.0",
        "compliance_frameworks": ["soc2", "iso27001"]
      },
      "internalParameters": {
        "aldeci_version": "2.1.0",
        "evidence_policy": "default"
      },
      "resolvedDependencies": [
        {
          "uri": "git+https://github.com/example/my-app@abc123",
          "digest": {"sha1": "abc123def456..."}
        }
      ]
    },
    "runDetails": {
      "builder": {
        "id": "https://github.com/actions/runner/v2.311.0"
      }
    }
  }
}
```

```

    },
    "metadata": {
      "invocationId": "https://github.com/example/my-app/actions/runs/123456",
      "startedOn": "2025-01-26T10:00:00Z",
      "finishedOn": "2025-01-26T10:30:00Z"
    },
    "byproducts": [
      {
        "name": "sbom.json",
        "digest": {"sha256": "f6e5d4c3b2a1..."}
      },
      {
        "name": "sarif.json",
        "digest": {"sha256": "1a2b3c4d5e6f..."}
      }
    ]
  }
}

```

### 1.7.3 4.3 Provenance Generation

AlDeci automatically generates provenance attestations for all evidence bundles:

```

# Automatic provenance generation
from services.provenance.attestation import generate_slsa_provenance

provenance = generate_slsa_provenance(
    subject_name="evidence-20250126-abc123.zip",
    subject_digest="a1b2c3d4e5f6...",
    builder_id="https://github.com/actions/runner/v2.311.0",
    source_uri="git+https://github.com/example/my-app@abc123",
    source_digest="abc123def456...",
    build_config={
        "component": "my-app",
        "version": "v2.1.0",
        "compliance_frameworks": ["soc2", "iso27001"]
    },
    materials=[
        {"uri": "sbom.json", "digest": "f6e5d4c3b2a1..."},
        {"uri": "sarif.json", "digest": "1a2b3c4d5e6f..."}
    ]
)

# Sign provenance
signed_provenance = sign_provenance(provenance, signing_key)

```

```
# Include in evidence bundle
bundle.add_file("provenance_attestation.json", signed_provenance)
```

## 1.8 5. Retention and Governance

### 1.8.1 5.1 Retention Policies

**Regulatory Requirements:** - SOC 2: 7 years - ISO 27001: 3 years (recommended 7 years) - APRA CPS 234: 7 years - GDPR: 6 years (UK), varies by jurisdiction - HIPAA: 6 years

**AlDeci Default:** 7 years (2555 days)

#### Configuration:

```
# config/evidence.yml
evidence:
  retention_days: 2555 # 7 years
  storage:
    backend: s3
    bucket: evidence-bundles
    region: ap-southeast-2
    encryption: AES-256
    lifecycle_policy: glacier_after_90_days
  cleanup:
    enabled: true
    schedule: "0 0 * * 0" # Weekly on Sunday
    dry_run: false
```

### 1.8.2 5.2 Storage Architecture

#### Tiered Storage:

##### Evidence Lifecycle

| Hot         | Warm          | Cold        | Archive     |
|-------------|---------------|-------------|-------------|
| (0-90 days) | (90-365 days) | (1-7 years) | (> 7 years) |
| S3 Standard | S3 IA         | S3 Glacier  | Delete      |
| \$0.023/GB  | \$0.0125/GB   | \$0.004/GB  | -           |

#### S3 Lifecycle Policy:

```
{
  "Rules": [
    {
      "Id": "evidence-lifecycle",
```

```

    "Status": "Enabled",
    "Transitions": [
      {
        "Days": 90,
        "StorageClass": "STANDARD_IA"
      },
      {
        "Days": 365,
        "StorageClass": "GLACIER"
      }
    ],
    "Expiration": {
      "Days": 2555
    }
  }
}
]
}

```

### 1.8.3 5.3 Access Control

#### IAM Policy (AWS):

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AlDeciEvidenceWrite",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:role/aldeci-evidence-writer"
      },
      "Action": [
        "s3:PutObject",
        "s3:PutObjectAcl"
      ],
      "Resource": "arn:aws:s3:::evidence-bundles/*"
    },
    {
      "Sid": "AuditorEvidenceRead",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:role/auditor"
      },
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
    }
  ]
}

```

```

    "Resource": [
      "arn:aws:s3::evidence-bundles",
      "arn:aws:s3::evidence-bundles/*"
    ]
  },
  {
    "Sid": "DenyDelete",
    "Effect": "Deny",
    "Principal": "*",
    "Action": [
      "s3:DeleteObject",
      "s3:DeleteObjectVersion"
    ],
    "Resource": "arn:aws:s3::evidence-bundles/*"
  }
]
}

```

## 1.9 6. CI/CD Integration

### 1.9.1 6.1 GitHub Actions

```

# .github/workflows/evidence.yml
name: Generate Evidence

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  evidence:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Generate SBOM
        run: |
          curl -sSfL https://raw.githubusercontent.com/anchore/syft/main/install.sh | sh -s --
          syft . -o cyclonedx-json > sbom.json

      - name: Run SAST
        run: |
          pip install semgrep

```



```

semgrep --config=auto --sarif > sarif.json

- name: Push to AlDeci
  env:
    ALDECI_TOKEN: ${ secrets.ALDECI_TOKEN }
  run: |
    curl -X POST https://api.devops.ai/products/aldeci/inputs/sbom \
      -H "Authorization: Bearer $ALDECI_TOKEN" \
      -F "file=@sbom.json" \
      -F "component=${ github.repository }" \
      -F "version=${ github.sha }"

    curl -X POST https://api.devops.ai/products/aldeci/inputs/sarif \
      -H "Authorization: Bearer $ALDECI_TOKEN" \
      -F "file=@sarif.json" \
      -F "component=${ github.repository }" \
      -F "version=${ github.sha }"

- name: Generate evidence bundle
  env:
    ALDECI_TOKEN: ${ secrets.ALDECI_TOKEN }
  run: |
    curl -X POST https://api.devops.ai/products/aldeci/evidence/generate \
      -H "Authorization: Bearer $ALDECI_TOKEN" \
      -H "Content-Type: application/json" \
      -d '{
        "component": "${ github.repository }",
        "version": "${ github.sha }",
        "compliance_frameworks": ["soc2", "iso27001"],
        "metadata": {
          "build_url": "${ github.server_url }/${ github.repository }/actions/runs/$
          "commit_message": "${ github.event.head_commit.message }",
          "author": "${ github.actor }"
        }
      }' \
      -o evidence-bundle.zip

- name: Upload evidence bundle
  uses: actions/upload-artifact@v3
  with:
    name: evidence-bundle
    path: evidence-bundle.zip

```

## 1.9.2 6.2 GitLab CI

```

# .gitlab-ci.yml
stages:
  - build
  - scan
  - evidence

build:
  stage: build
  script:
    - npm run build
  artifacts:
    paths:
      - dist/

scan:
  stage: scan
  script:
    - syft . -o cyclonedx-json > sbom.json
    - semgrep --config=auto --sarif > sarif.json
  artifacts:
    paths:
      - sbom.json
      - sarif.json

evidence:
  stage: evidence
  script:
    - |
      curl -X POST https://api.devops.ai/products/aldecideci/inputs/sbom \
        -H "Authorization: Bearer $ALDECI_TOKEN" \
        -F "file=@sbom.json" \
        -F "component=$CI_PROJECT_PATH" \
        -F "version=$CI_COMMIT_SHA"

    - |
      curl -X POST https://api.devops.ai/products/aldecideci/inputs/sarif \
        -H "Authorization: Bearer $ALDECI_TOKEN" \
        -F "file=@sarif.json" \
        -F "component=$CI_PROJECT_PATH" \
        -F "version=$CI_COMMIT_SHA"

    - |
      curl -X POST https://api.devops.ai/products/aldecideci/evidence/generate \
        -H "Authorization: Bearer $ALDECI_TOKEN" \
        -H "Content-Type: application/json" \
        -d "{
        \component\": \"$CI_PROJECT_PATH\",

```

```

    \"version\": \"$CI_COMMIT_SHA\",
    \"compliance_frameworks\": [\"soc2\", \"iso27001\"],
    \"metadata\": {
      \"build_url\": \"$CI_PIPELINE_URL\",
      \"commit_message\": \"$CI_COMMIT_MESSAGE\",
      \"author\": \"$GITLAB_USER_EMAIL\"
    }
  }" \
  -o evidence-bundle.zip
artifacts:
  paths:
    - evidence-bundle.zip

```

## 1.10 7. Compliance Framework Mapping

### 1.10.1 7.1 SOC 2 Type II

| Control                          | Evidence Type  | AlDeci Artifact              |
|----------------------------------|----------------|------------------------------|
| CC6.1 - Logical access controls  | Access logs    | IAM policies, RBAC config    |
| CC6.6 - Vulnerability management | Scan results   | SARIF findings, risk reports |
| CC7.1 - Change management        | Build metadata | Provenance attestations      |
| CC7.2 - System monitoring        | Observability  | Metrics, traces, logs        |
| CC8.1 - Risk assessment          | Risk scores    | FixOpsRisk reports           |

### 1.10.2 7.2 ISO 27001

| Control                                | Evidence Type      | AlDeci Artifact               |
|----------------------------------------|--------------------|-------------------------------|
| A.12.6.1 - Vulnerability management    | Scan results       | SARIF findings, risk reports  |
| A.14.2.1 - Secure development policy   | SAST results       | SARIF findings (SAST)         |
| A.14.2.5 - Secure system engineering   | SBOM               | Normalized SBOM, dependencies |
| A.18.2.3 - Technical compliance review | Compliance reports | Policy evaluations            |

### 1.10.3 7.3 APRA CPS 234

| Requirement                        | Evidence Type        | AlDeci Artifact                  |
|------------------------------------|----------------------|----------------------------------|
| Req 17 - Vulnerability management  | Scan results         | SARIF findings, risk reports     |
| Req 18 - Patch management          | Version lag          | SBOM with version lag analysis   |
| Req 28 - Control testing           | Test results         | Scan frequency, coverage metrics |
| Req 31 - Vulnerability remediation | Remediation tracking | MTTR metrics, Jira tickets       |

## 1.11 8. Implementation Examples

### 1.11.1 8.1 Example: Microservices Architecture

**Scenario:** 50 microservices, each with independent CI/CD pipelines

**Solution:** Centralized evidence collection with per-service evidence bundles

```
# evidence-config.yml (per service)
service:
  name: user-service
  version: v2.1.0
  compliance_frameworks:
    - soc2
    - iso27001
  evidence_requirements:
    - type: sbom
      format: cyclonedx
      frequency: every_build
    - type: sarif
      scan_types: [SAST, SCA]
      frequency: every_build
    - type: container_scan
      scan_types: [vulnerability, malware]
      frequency: every_build
  retention_days: 2555
```

#### CI/CD Integration:

```
# Each service pipeline
syft . -o cyclonedx-json > sbom.json
semgrep --config=auto --sarif > sarif.json
trivy image --format sarif user-service:v2.1.0 > container-scan.sarif

# Push to AlDeci
aldecideci push sbom sbom.json --service user-service --version v2.1.0
aldecideci push sarif sarif.json --service user-service --version v2.1.0
```

```
aldecideci push sarif container-scan.sarif --service user-service --version v2.1.0
```

```
# Generate evidence bundle
```

```
aldecideci evidence generate --service user-service --version v2.1.0 --output evidence-bundle.zip
```

### 1.11.2 8.2 Example: Monorepo

**Scenario:** Single monorepo with multiple applications

**Solution:** Component-level evidence generation with shared compliance configuration

```
# .aldecideci/config.yml (monorepo root)
```

```
monorepo:
  name: my-monorepo
  components:
    - name: frontend
      path: apps/frontend
      compliance_frameworks: [soc2]
    - name: backend
      path: apps/backend
      compliance_frameworks: [soc2, iso27001]
    - name: worker
      path: apps/worker
      compliance_frameworks: [soc2]
  shared_evidence:
    - type: repository_scan
      frequency: daily
    - type: dependency_scan
      frequency: weekly
```

#### CI/CD Integration:

```
# Generate evidence for each component
```

```
for component in frontend backend worker; do
```

```
  syft apps/$component -o cyclonedx-json > sbom-$component.json
```

```
  semgrep --config=auto apps/$component --sarif > sarif-$component.json
```

```
  aldecideci push sbom sbom-$component.json --component $component --version $COMMIT_SHA
```

```
  aldecideci push sarif sarif-$component.json --component $component --version $COMMIT_SHA
```

```
done
```

```
# Generate monorepo-level evidence
```

```
aldecideci evidence generate-monorepo --config .aldecideci/config.yml --version $COMMIT_SHA
```

## 1.12 9. Best Practices

### 1.12.1 9.1 Evidence Generation

1. **Generate evidence on every build** - Don't wait for audits
2. **Include build metadata** - Commit SHA, build URL, author
3. **Sign immediately** - Don't delay signing
4. **Validate before storing** - Check evidence completeness
5. **Use consistent naming** - evidence-YYYYMMDD-{id}.zip

### 1.12.2 9.2 Key Management

1. **Rotate keys annually** - Maintain old keys for verification
2. **Use hardware security modules (HSM)** - For production keys
3. **Publish public keys** - Make verification easy for auditors
4. **Separate signing keys** - Different keys for different purposes
5. **Audit key usage** - Log all signing operations

### 1.12.3 9.3 Storage and Retention

1. **Use tiered storage** - Hot/warm/cold based on age
2. **Enable versioning** - Protect against accidental deletion
3. **Replicate across regions** - Disaster recovery
4. **Encrypt at rest** - AES-256 or better
5. **Automate lifecycle** - Don't rely on manual cleanup

### 1.12.4 9.4 Compliance Mapping

1. **Document control mappings** - Link evidence to requirements
  2. **Maintain evidence matrix** - Track coverage gaps
  3. **Update mappings regularly** - As frameworks evolve
  4. **Include rationale** - Explain why evidence satisfies control
  5. **Get auditor buy-in** - Validate approach before audit
- 

## 1.13 10. Troubleshooting

### 1.13.1 10.1 Common Issues

**Issue:** Evidence bundle generation fails with “missing SBOM”

**Solution:**

```
# Verify SBOM was pushed
curl -X GET https://api.devops.ai/products/aldeci/inputs/sbom/list \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"component": "my-app", "version": "v2.1.0"}'
```

```
# If missing, push SBOM
curl -X POST https://api.devops.ai/products/aldeci/inputs/sbom \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@sbom.json" \
  -F "component=my-app" \
  -F "version=v2.1.0"
```

**Issue:** Signature verification fails

**Solution:**

```
# Check public key URL
curl -I https://public-keys.s3.amazonaws.com/aldeci/evidence-signing-key.pub

# Download correct public key
wget https://public-keys.s3.amazonaws.com/aldeci/evidence-signing-key.pub

# Verify with correct key
openssl dgst -sha256 -verify evidence-signing-key.pub \
  -signature signatures/manifest.sig \
  manifest.yaml
```

**Issue:** Evidence bundle too large (> 100 MB)

**Solution:**

```
# Exclude large files from evidence bundle
evidence:
  bundle_contents:
    - normalized_sbom
    - risk_report
    - sarif_findings # Exclude raw SARIF (can be large)
    - provenance_attestation
    - policy_evaluations
    - manifest
  compression: gzip # Enable compression
  max_size_mb: 100
```

---

**DevOps.ai** | Sydney, Australia | <https://devops.ai> | [contact@devops.ai](mailto:contact@devops.ai)

© 2025 DevOps.ai. All rights reserved.